

# Краткая схема решения

GigaCode CLI для реверсивной аналитики кода

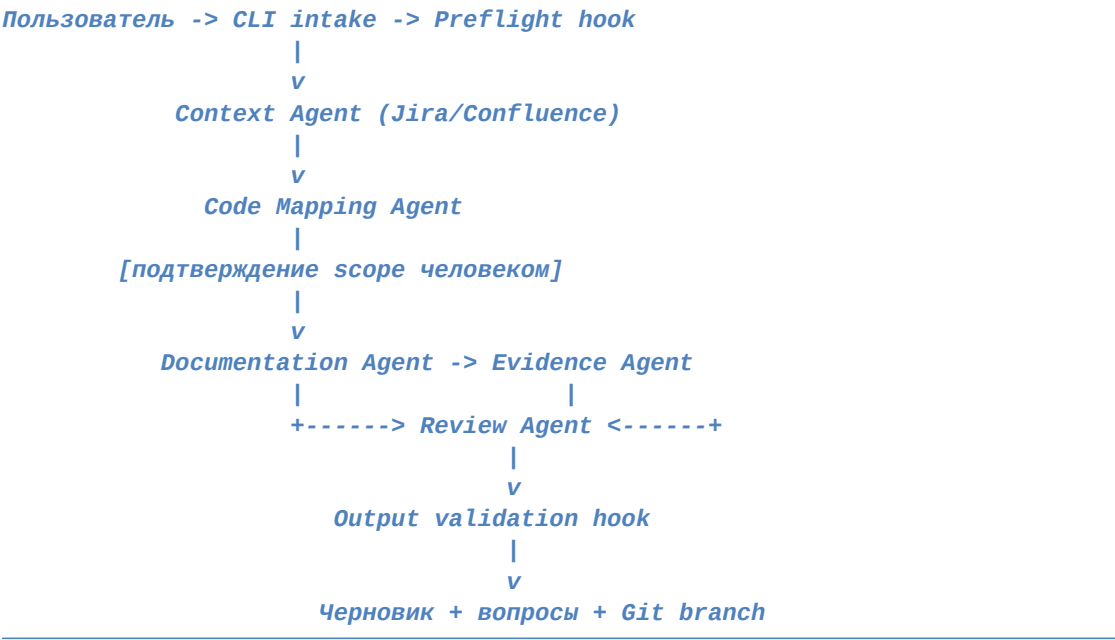
## 1. Идея в одном абзаце

Решение превращает реверсивную аналитику одной бизнес-фичи в управляемый рабочий контур: CLI собирает входные данные, картирует код, вытягивает контекст из Jira/Confluence, формирует черновик документации рядом с кодом и явно отделяет подтверждённые факты от пробелов и вопросов.

## 2. Что входит в v1

| Компонент                                   | Назначение                                                                       |
|---------------------------------------------|----------------------------------------------------------------------------------|
| 5 субагентов                                | Разделение ответственности без перегрузки оркстратора                            |
| 1 общий skill                               | Единые правила reverse-analysis: источники, шаблон, вопросы, запреты             |
| 2 hook-a                                    | Preflight для проверки входных данных и output validation для проверки артефакта |
| Utilities: repomix, graphify, Atlassian MCP | Карта репозитория, граф зависимостей, внешние источники                          |
| Документация в репозитории                  | Markdown-артефакты рядом с кодом, версия в Git                                   |

## 3. Основной флоу



Ключевая логика: сначала score и карта кода, потом текст. Не наоборот.

## 4. Субагенты

| Агент          | Вход                      | Выход / задача                                       |
|----------------|---------------------------|------------------------------------------------------|
| Intake & Scope | feature, Jira, Confluence | Уточняет границы, фиксирует что включено и исключено |

|                |                                  |                                                                       |
|----------------|----------------------------------|-----------------------------------------------------------------------|
| Code Mapping   | scope + repo map + graph         | Находит точки входа, цепочки вызовов, интеграции, список файлов       |
| Documentation  | карта кода + релевантные файлы   | Пишет overview/flow/integrations/data/questions по шаблону            |
| Evidence & Gap | черновик + код                   | Проверяет утверждения, строит матрицу доказательств и список пробелов |
| Review         | черновик + матрица доказательств | Проверяет шаблон, термины, стиль, не добавляя новых фактов            |

## 5. Skill, hooks и инструменты

- Skill reverse-analysis задаёт инварианты: обязательные вводные, приоритет источников, правила вопросов пользователю, шаблон документа, запрет на домысливание.
- Preflight hook проверяет наличие business feature, Jira/Confluence или явного отказа от них, целевой каталог, шаблон и корректную ветку.
- Output validation hook проверяет наличие всех файлов, обязательных секций, блока открытых вопросов и отсутствие маркеров незавершённости.
- repomix даёт структуру репозитория; graphify помогает выделить минимальный подграф функции; Atlassian MCP подтягивает Jira/Confluence.

## 6. Артефакты в репозитории

```
docs/
├── features/
│   └── <feature-name>/
│       ├── overview.md
│       ├── flow.md
│       ├── integrations.md
│       ├── data.md
│       └── questions.md
rules/
├── reverse-analysis.md
├── branch-naming.md
hooks/
├── preflight_check.py
└── validate_output.py
```

---

## 7. Что важно для пилота

- Единица анализа — одна бизнес-фича, а не весь сервис.
- Сначала граф и список файлов, затем загрузка релевантного кода.
- Неподтверждённое — в assumptions или questions, а не в основной текст.
- Критические решения остаются за человеком: границы фичи, противоречия между кодом и Jira, принятие пробелов.
- Оптимальный старт — 2-3 функции средней сложности.

## 8. Вывод

В версии v1 достаточно 5 субагентов, 1 общего skill и 2 hook-ов. Это даёт управляемый контур без лишней автоматизации и позволяет быстро проверить идею на пилоте.